CS 0007: Introduction to Java Lecture 10

Nathan Ong
University of Pittsburgh
October 4, 2016

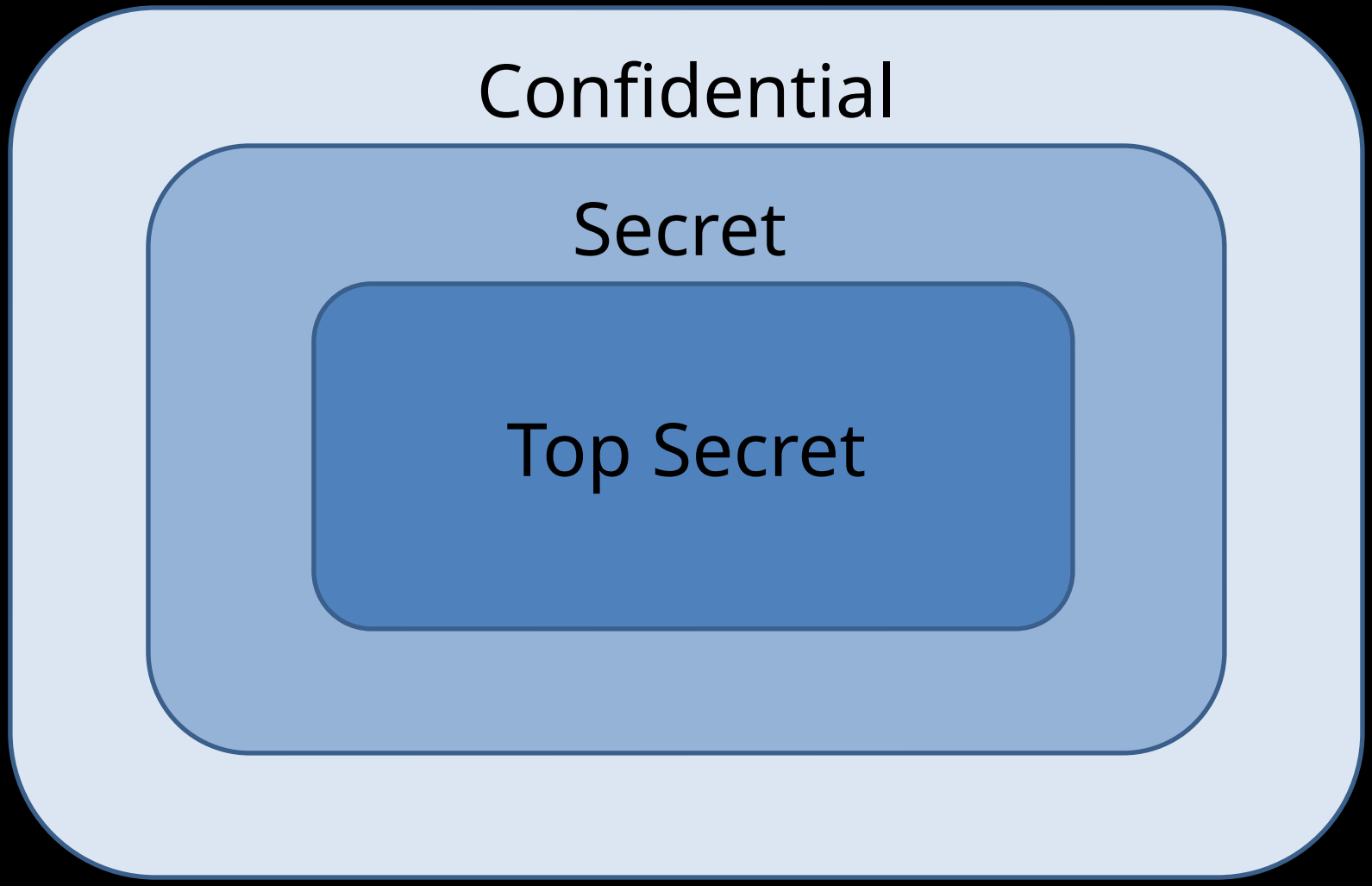
`if()` recap

```
//always run this code
if(condition1 is true)
{
	//run this code
}
else if(condition2 is true)
{
	//run this code
}
else if(condition3 is true)
{
	...
}
...
else //if conditions 1...n were not satisfied
{
	//run this code
}
//always run this code
```

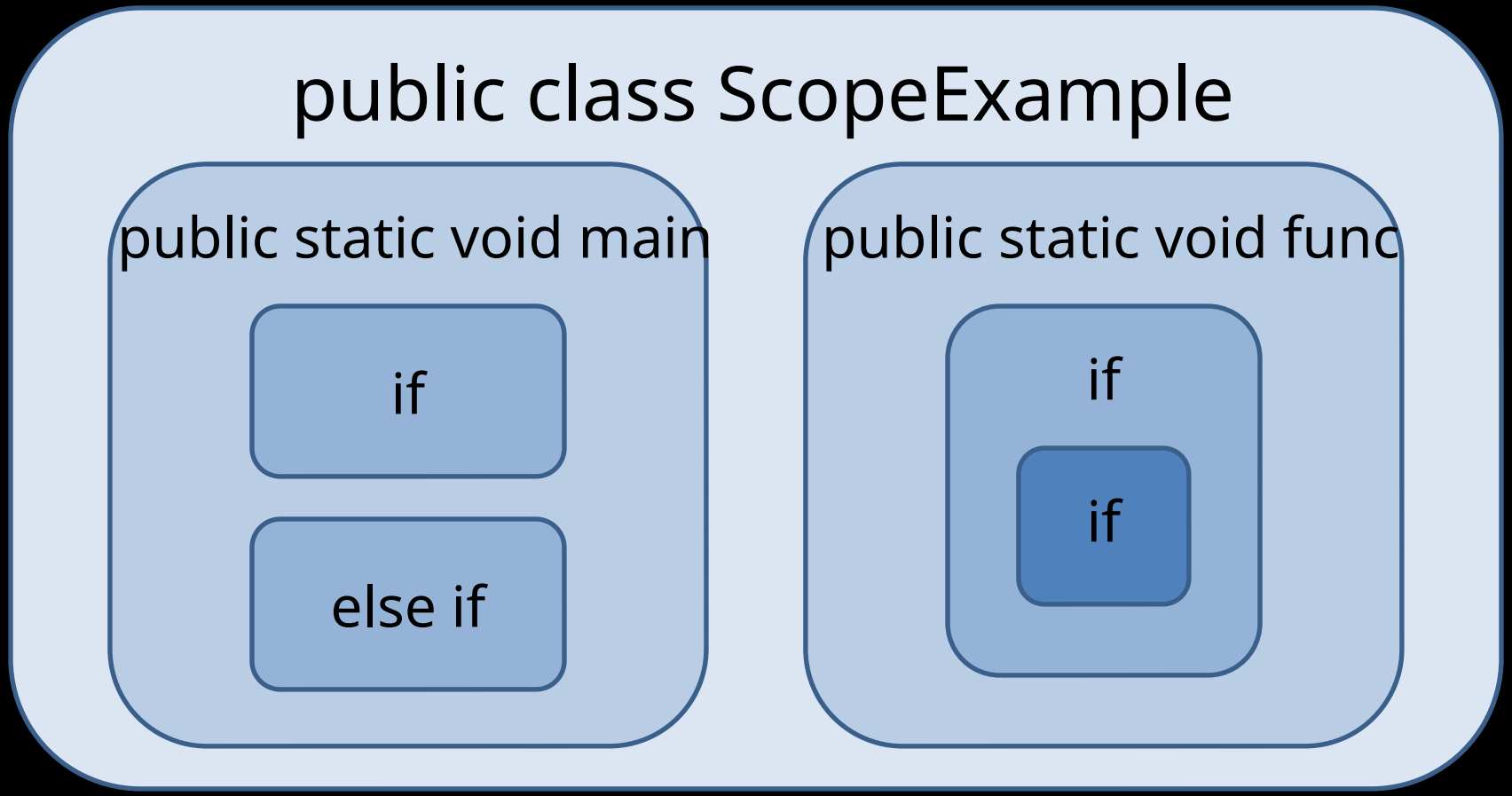
Revisiting Scope

- Scope works by curly brackets.
- Curly brackets surround all newly declared information, keeping it hidden from code outside of the curly brackets.

Scope



Scope



Darker shades can see lighter shades, but not vice versa

Scope in Code

```
public class Name
{
  public static void main(String[] args)
  {
  int x = 0;

  //x == 0, y is undefined
  if(x == 0)
  {
  int y = 10;
  x = 15;
  //x == 15, y == 10
  }
  //x == 15, y is undefined
  }//end method main
}//End class Name
```

A Special Annoying Case

```
Scanner scanner = new Scanner(System.in);
int numTrees = scanner.nextInt();
if(numTrees == 0)
{
 System.out.println("Desert");
}
else if(numTrees == 1)
{
 System.out.println("Island");
}
else if(numTrees == 2)
{
 System.out.println("Beach");
}
else if(numTrees == 3)
{
 System.out.println("Forest");
}
else
{
 System.out.println("Jungle");
}
```

Switch-Case

- A Switch-Case statement is like a long if, else-if, else block concerning only one variable and several possible discrete values (not ranges).
- This increases readability, but they act similarly.
- Works for `byte`, `char`, `short`, `int`, and `String`. (Any others out of scope)
- For every `case` possibility, you need a `break` statement, unless you wanted to combine several cases together.

A Special Annoying Case

```
Scanner scanner = new Scanner(System.in);
int numTrees = scanner.nextInt();
switch(numTrees)
{
 case 0:
 System.out.println("Desert");
 break;
 case 1:
 System.out.println("Island");
 break;
 case 2:
 System.out.println("Beach");
 break;
 case 3:
 System.out.println("Forest");
 break;
 default:
 System.out.println("Jungle");
 break;
}
```

ARRAYS, LOOPS, AND RECURSION

A Sudden Need

- I need a function that returns the first 100 terms of the harmonic series.
- Harmonic series:

$$\sum_{n=1}^{\infty} \frac{1}{n} = 1 + \frac{1}{2} + \frac{1}{3} + \dots$$

The Dumb Way

```
public static void main(String[] args)
{
  double term0 = 1.0;
  double term1 = 1.0/2.0;
  double term2 = 1.0/3.0;
  …
}
```

Lists

- How do we do lists?
- Using 100 variables as a list is not scalable! What if we need more? What if I change the names? What if I wanted a different series?
- A WILD STRUCTURE APPEARS

Arrays

- Arrays are ordered lists of things depending on the type you specify
- How do you make one?

```
Type[] name = new Type[size];
```

```
int[] empty = new int[10];
```

```
Type[] name = values;
```

```
int[] list = {1,2,3,4};
```

Now What?

- I want the first element
- `list[0]`
- I want the last element
- `list[3]`
- I want the length
- `list.length == 4`
- All operations for `int` are allowed for an individual element
- list[0] + list[2] == list[3]

How do I get every element so I can print it?

- Reference every element
- Many if statements!!!!!! YAAAAAAAAYYYY
- No please don't
- But what else can we do?
- WILD KEYWORD APPEARS

while(){…}

- While a condition is true, run the block
- While not at end of the list, print out the next element

```
while(not at end of list)
{
        System.out.println(next element);
}//end loop while(not at end of list)
```

not at end of list

- How do we know we're at the end of the list?
- `list.length`
- How do we know which one we're at?
  - We need to keep track

```
int currElement = 0;
```

  - starts at 0
  - not equal to the end!

```
currElement != list.length
```

while(){…}

- While not at the end of the list, print out the next element

```
int currElement = 0;
while(currElement != list.length)
{
 System.out.println(<next element>);
}//end loop while(currElement!=list.length)
```